

조기취업형 계약학과

반도체 전공 세미나

학업 + 현장을 함께

2026.03



Claude Code

실전 워크플로우 & 고급 자동화

반도체 전공 계약학과생을 위한 설치부터 Hooks·Sub-Agent·Skills까지 — 120분 완성

00 설치 & 환경

01 컨텍스트

02 워크플로우

03 Skills

04 Sub-Agent

05 Hooks

2 회차

00

오늘 세미나 목표 & 타임라인

강연 60분 + 실습 60분 = 총 120분 / 노트북 필수 지참

환경 설치

Claude Code 설치
환경 완전 셋업

컨텍스트 제어

토큰 절약-CLAUDE.md
구조화 전략

Skills 자동화

SKILL.md 작성
팀 공유 시스템

Sub-Agent 병렬

동시 실행으로
3배 빠른 작업

Hooks 자동화

이벤트 기반
자동 워크플로우

00

1회차 핵심 복습 (30초 리마인드)

2회차는 '제어'입니다 — 1회차는 Claude를 '쓰는 법', 2회차는 '부리는 법'

✓ 프롬프트 3요소 공식

역할 + 맥락 + 형식 = 좋은 답변

✓ Skills 개념

SKILL.md = 재사용 가능한 업무 매뉴얼

✓ 보안 원칙

기밀 데이터 입력 금지 / 결과물 검증 필수

오늘 달라지는 것

대상	Claude 입문자	→ Claude Code 실사용자
준비물	스마트폰 OK	→ 노트북 필수 (Claude Code 설치)
핵심 메시지	AI를 쓰는 법	→ AI를 제어·자동화하는 법
실습 수준	프롬프트 입력	→ 파일 생성·설정·스크립트 작성



환경 설치 & 셋업

Claude Code 설치부터 첫 실행까지 — 15분

설치

시작 전 준비물 확인

아래 항목이 모두 준비되어야 실습 참여 가능합니다

Node.js 18 이상

확인: `node --version`

✓ v18.x 이상이면 OK

💡 nodejs.org 에서 LTS 버전 설치

터미널 (Windows)

확인: PowerShell / cmd

✓ 윈도우 기본 탑재

💡 Windows Terminal 설치 권장

VS Code

확인: `code --version`

✓ 아무 버전이나 OK

💡 code.visualstudio.com

Claude.ai 계정

확인: claude.ai 접속

✓ Pro 이상 권장

💡 무료 계정도 일부 기능 사용 가능

Claude Code 설치 방법

npm 한 줄로 전 세계 어디서든 설치 — Windows / Mac / Linux 동일

1. Claude Code 전역 설치 (터미널/PowerShell에서 실행)

```
$ npm install -g @anthropic-ai/claude-code
```

2. 설치 확인

```
$ claude --version
```

→ Claude Code v2.1.xx 출력되면 성공!

3. 로그인 (claude.ai 계정)

```
$ claude login
```

→ 브라우저 자동 오픈

→ Claude.ai 로그인 후 승인

→ 터미널에 ✓ Logged in 표시

4. 첫 실행 테스트

```
$ cd 프로젝트폴더
```

```
$ claude
```

→ > 프롬프트 나타나면 성공!

종료: /exit 또는 Ctrl+C

— 반도체 계약학과 테스트 —

연구 폴더에서 바로 실행:

```
$ cd ~/semiconductor-lab
```

```
$ claude
```

> 안녕! 나는 반도체 공정

실험 노트를 관리하고 싶어.

설치

Claude Code 인터페이스 & 필수 명령어

처음 보면 낯설지만 — 5개만 알면 바로 시작 가능

일반 모드

기본값

기본 대화 모드
파일 읽기·수정 가능

Plan Mode

Shift+Tab

계획만 세우고 실행 안 함
큰 작업 전 반드시 사용

Auto Mode

--dangerously

승인 없이 자동 실행
신뢰할 때만 사용

주요 명령어

<code>/clear</code>	컨텍스트 완전 초기화	새 작업 시작 시
<code>/compact</code>	대화 압축 (맥락 유지)	컨텍스트 절반 줄이기
<code>/memory</code>	메모리 확인·편집	학습 내용 관리
<code>/context</code>	토큰 사용량 확인	낭비 체크
<code>/exit</code>	Claude Code 종료	세션 종료

핵심 파일 구조 & settings.json

이 구조만 이해하면 Skills·Hooks·Agent 모두 활용 가능

Claude Code 폴더 구조

~/ .claude/	# 개인 전역 설정
settings.json	# 혹은 권한 설정
skills/	# 개인 스킬
ppt-gen/SKILL.md	
agents/	# 개인 에이전트
.claude/	# 프로젝트 설정
settings.json	# 프로젝트 혹은
skills/	# 팀 공유 스킬
agents/	# 팀 에이전트
CLAUDE.md	# 프로젝트 지시

settings.json 기본 구조

```
{
  "permissions": {
    "allow": ["Bash(*)"],
    "deny": []
  },

  "hooks": {
    "Stop": [...],
    "PostToolUse": [...]
  },

  "model": "claude-sonnet-4-6"
}
```


0

1

컨텍스트 관리 전략

토큰을 아끼면 성능이 오른다 — 12분

CLAUDE.md Lazy Loading 전략

모든 내용을 CLAUDE.md에 넣으면 매 세션마다 수천 토큰 낭비

❌ 나쁜 예 - 직접 작성 (토큰 낭비)

CLAUDE.md

반도체 공정 스펙

CVD 온도: 300~900°C / 압력: 1~760 Torr

PVD 진공도: 10⁻⁶ Torr 이하

ALD 사이클 횟수: 50~500회

... (50개 파라미터 더)

측정 장비 목록 (표 30개)

...

✅ 좋은 예 - 참조만 (Lazy Loading)

CLAUDE.md

프로젝트 문서

- 공정 스펙: @docs/process-spec.md

- 장비 목록: @docs/equipment.md

- 측정 규칙: @docs/measurement.md

- 코딩 규칙: @docs/conventions.md

Claude는 필요할 때만 파일 읽음!

💡 폴더별 CLAUDE.md 분리: src/cvd/CLAUDE.md · src/pvd/CLAUDE.md — 해당 폴더 작업 시에만 로드되어 컨텍스트 bloat 방지

/memory & 한 세션 = 한 피처 원칙

/memory로 Second Brain 구축 + 컨텍스트 윈도우 전략

```
# /memory 사용법
# 1. 중요한 것을 발견하면:
> 이 내용 기억해줘:
  ALD 공정에서 TMA 펄스는
  0.1s / 퍼지는 5s 최적이야

# 2. 저장 확인:
> /memory
# → MEMORY.md 편집창 열림

# 저장위치:
# ~/.claude/projects/<proj>
#           /memory/MEMORY.md
```

```
# 한 세션 = 한 피처 워크플로우
# Step 1: Plan Mode에서 설계
> Shift+Tab → Plan Mode 전환
> 전체 작업을 단계별로 설계

# Step 2: 첫 번째 단계만 실행
> /clear # 컨텍스트 초기화
> 1단계 작업 지시

# Step 3: 완료 → 다음 단계 반복
```

/clear

컨텍스트 완전 삭제

새 피처 시작, 완전히 다른 작업으로 전환

/compact

매라 유지하며 아츠

가으 자어 게소 트크마 적야친구 시으 때

0

2

워크플로우 철학 & 코딩 패턴

Plan → TDD → 에러처리 → WAT — 13분

Plan Mode 먼저 + TDD 루프

Plan 없이 실행 = 재앙 / 작은 변경 → 테스트 → 커밋 반복이 핵심

Plan Mode 사용법 (Shift+Tab)

1. Plan Mode 진입

> Shift+Tab # 모드 전환
> 반도체 공정 데이터 분석
자동화 스크립트를 만들어줘

2. Claude가 계획 제시

→ 수정할 파일 목록
→ 접근 방식 설명
→ 예상 단계 수

3. 계획 검토 후 승인

> 좋아, 진행해줘

TDD 스마트 코딩 루프

Step 1: 작은 단위로 구현

> CVD 온도 파싱 함수
하나만 먼저 구현해줘

Step 2: 테스트 돌리기

> pytest tests/test_cvd.py -v

Step 3: 린트 체크

> ruff check . && mypy .

Step 4: 커밋

> git add . && git commit -m
'feat: add CVD temp parser'

→ 다음 기능으로 반복!

에러 처리 & TODO.md 활용

에러 로그 그대로 붙여넣기 + TODO.md로 세션 연속성 유지

```
# ❌ 나쁜 예 - 해석해서 전달
> 데이터 파싱할 때 뭔가 오류가..

# ✅ 좋은 예 - 스택트레이스 통째로
> 아래 에러 해결해줘:
Traceback (most recent call last):
  File 'parse_cvd.py', line 47
    in read_temperature
ValueError: invalid literal
for int() with base 10: 'N/A'
```

```
# TODO.md 하루 시작 루틴

# 1. TODO.md 작성
- [ ] CVD 온도 파싱 수정
- [ ] ALD 사이클 분석 스크립트
- [x] 데이터 정규화 (완료)

# 2. Claude에게 지시
> TODO.md 읽고 첫 번째 항목
부터 시작해줘
```

WAT 프레임워크

W	— Workflow	plain English로 단계 먼저 정의 → Plan Mode로 연결
A	— Agents	Self-healing + Sub-agent 병렬 처리로 3배 빠르게
T	— Tools	거대 스크립트 1개 ❌ → 작은 원자적 도구 여러 개 ✅

0

3

Skills 심화

SKILL.md 작성법 + 반도체 계약학과 맞춤 스킬 설계 — 10분

SKILL.md 완전 정복

description이 전부다 — 트리거 문장 3개 이상, 첫 줄에 핵심 기능

```
# 완성된 SKILL.md 예시 (반도체 실험 보고서 스킬)
```

```
---
```

```
name: experiment-report
```

```
description: "반도체 공정 실험 보고서 자동 생성.
```

```
  '실험보고서 만들어줘', '보고서 써줘',
```

```
  '실험 결과 정리해줘' 요청 시 자동 트리거."
```

```
---
```

```
## 목적
```

```
실험 데이터와 조건을 입력받아 전문적인 실험 보고서를 생성합니다.
```

```
## 절차
```

```
1. 실험 목적, 조건(온도/압력/시간), 결과 수치 확인
```

```
2. 결과 해석 및 오차 분석 자동 생성
```

```
3. 개선 방향 3가지 제안
```

```
4. Word(.docx) + PDF 두 파일 동시 생성
```

```
## 자체 검증 체크리스트
```

```
- [ ] 모든 수치에 단위가 표기되었는가?
```

```
- [ ] 참고문헌이 정확히 인용되었는가?
```


skill-creator 활용 + 반도체 계약학과 추천 스킬

skill-creator가 SKILL.md를 자동 작성 — '스킬 만들어줘' 한 마디면 끝

```
# skill-creator 설치 & 사용  
$ /install-plugin skill-creator
```

```
# 설치 후 바로 사용:  
> 반도체 실험 보고서 만드는  
스킬 만들어줘  
# → SKILL.md 자동 생성!
```

```
# 스킬 저장 위치  
# 개인용 (~/.claude/skills/)  
# 팀 공유 (.claude/skills/)  
$ git add .claude/skills/  
$ git commit -m 'add exp-report'
```

experiment-report

'보고서 만들어줘'

공정 실험 보고서 자동화

datasheet-translator

'스펙 번역해줘'

영문 데이터시트 한국어 변환

paper-summarizer

'논문 요약해줘'

IEEE/arXiv 논문 3줄 요약

patent-analyzer

'특허 분석해줘'

반도체 특허 청구항 분석

lab-notes

'실험노트 정리'

측정 데이터 → 체계적 노트

email-draft

'이메일 써줘'

교수님/현장 보고 이메일

0

4

Sub-Agent & 병렬 실행

심부름꾼에게 맡기고 결과만 받기 — 10분

Sub-Agent 개념 & 내장 5종

메인 Claude 안에서 별도 작업 공간을 가진 도우미 — 컨텍스트 분리가 핵심

에이전트

모델

권한

용도

Explore

Haiku

읽기전용

코드 탐색·파일 검색·구조 파악

Plan

상속

읽기전용

Plan Mode에서 계획 수립 연구

General

상속

모든 도구

탐색+수정 복잡한 다단계 작업

Bash

상속

Bash만

별도 컨텍스트 터미널 명령 실행

Guide

Haiku

읽기전용

Claude Code 기능 질문 답변

커스텀 에이전트 만들기 & 병렬 패턴

/agents 명령어로 5단계 마법사 — data-processor 실전 예시

```
# 커스텀 에이전트 YAML 구조
---
name: data-processor
description: "반도체 측정 데이터 처리
  전문가. CSV 파싱. 이상값 탐지.
  시각화 자동 생성."
tools: Read, Write, Bash
model: sonnet
---
```

```
# 호출 시 자동으로:
1. CSV 파일 구조 분석
2. 이상값 자동 탐지
3. 그래프 PNG 생성
```

병렬 실행 실전 패턴

```
# 반도체 연구 병렬 처리:
> Sub-Agent로 다음을 병렬 처리:
- CVD 실험 데이터 파싱 & 분석
- 해당 공정의 논문 요약 (3줄)
- 교수님 보고용 PPT 초안 작성
```

결과: 순차 20분 → 병렬 7분

```
✓ 독립적 작업 병렬 분배
✓ 명확한 역할. 범위 지정
✗ 의존성 작업 무리한 병렬화
```

```
# ⚠ 주의:
# Sub-Agent끼리 직접 통신 불가
# 의존성 있는 작업은 순차로!
```

0

5

Hooks 자동화 엔진

이벤트 발생 → 자동 실행 — 10분

Hook JSON 구조 & 4가지 이벤트

이벤트 → matcher 조건 검사 → 액션 자동 실행 — Claude는 기다림

settings.json Hook 기본 구조

```
{
  "hooks": {
    "Stop": [
      {
        "matcher": "*",
        "hooks": [
          {
            "type": "command",
            "command": "알림 명령어"
          }
        ]
      }
    ]
  }
}
```

PreToolUse

□ 도구 호출 직전

→ 입력값 검증·승인·차단

PostToolUse

□ 도구 실행 직후

→ 린트·포맷·후처리

Notification

□ 사용자 응답 대기

→ 알림 전송·로깅

Stop

□ 에이전트 턴 종료

→ 보고서 생성·상태 저장

반도체 계약학과 맞춤 Hook 레시피 3종

'Hook 만들어줘' 한 마디면 settings.json에 자동 추가

```
# Hook 1: 작업 완료 Windows 알림
{"Stop":[{"matcher":"*", "hooks":[{"type":"command",
  "command":"powershell -c \"
    [console]::beep(1000,500);
    Add-Type -AssemblyName
    PresentationFramework;
    [Windows.MessageBox]::Show(
    'Claude 작업 완료!')\"
  }]}]}
```

```
# Hook 2: 파일 저장 후 자동 린트
{"PostToolUse":[{"matcher":"Write",
  "hooks":[{"type":"command",
    "command":"cd $PROJECT_ROOT &&
    ruff check --fix . &&
    echo '✓ 린트 완료'\"
  }]}]}
```

```
# Hook 3: 위험 명령어 차단 (PreToolUse)
{"PreToolUse":[{"matcher":"Bash",
  "hooks":[{"type":"command",
    "command":"bash -c 'echo
    $CLAUDE_TOOL_INPUT |
    grep -q \"rm -rf\" &&
    echo BLOCKED && exit 1
    || exit 0'\"
  }]}]}
# → rm -rf 자동 차단! 보안 강화
```

⚠ Hook 실행 중 Claude는 멈추고 기다림 → timeout 필수 설정 / 무거운 작업은 백그라운드(&)로 실행

8단계 파이프라인 — WAT 시너지

연구 데이터 → 자동 처리 → 보고서 : 음성입력 → MCP → Sub-Agent → Hooks → 보고서



0

5

실습 세션 시작

설치 확인 → CLAUDE.md → Skills → Sub-Agent → Hooks — 60분

설치 확인 & 환경 점검

실습 시작 전 모든 참가자가 아래 명령어 실행 — 강사와 함께 확인

```
# ① Node.js 버전 확인
$ node --version
# v18.0.0 이상이면 OK

# ② Claude Code 설치
$ npm install -g @anthropic-ai/claude-code

# ③ 설치 확인
$ claude --version
# Claude Code v2.1.xx

# ④ 로그인
$ claude login
# 브라우저 열림 → 승인 → 완료

# ⑤ 테스트 실행
$ mkdir ~/semicon-lab && cd ~/semicon-lab
$ claude
# > 프롬프트 나타나면 성공! 🎉
```

! 오류 발생 시 해결법

오류 1: npm not found

nodejs.org 에서 LTS 설치
(Node 포함 npm 자동 설치됨)

오류 2: permission denied

```
sudo npm install -g \
  @anthropic-ai/claude-code
```

오류 3: login 실패

claude.ai Pro 계정 필요
브라우저 팝업 차단 해제

오류 4: 버전 오래됨

```
npm update -g \
  @anthropic-ai/claude-code
```

CLAUDE.md + /memory 실습

반도체 연구 프로젝트용 CLAUDE.md 작성 + 메모리 저장 체험 — 10분

```
# Step 1: 프로젝트 폴더 만들기
$ mkdir ~/semicon-project
$ cd ~/semicon-project
$ claude
```

```
# Step 2: CLAUDE.md 생성 요청
> 우리 연구실은 반도체 소자
  공정 연구를 해. CVD·PVD·ALD
  공정이 주요 연구 주제고,
  측정 장비는 SEM·XRD·AFM이야.
  Lazy Loading 구조로
  CLAUDE.md 만들어줘.
```

```
# Step 3: /memory 체험
> 기억해줘: ALD 공정에서
  TMA 펄스 0.1s / 퍼지 5s가
  최적 조건이야.
```

```
> /memory
# → MEMORY.md 편집창 열림
```

```
# Step 4: 새 세션에서 확인
> /clear
> ALD 최적 조건 알아?
# → TMA 0.1s / 퍼지 5s 기억함!
# 💡 세션 종료해도 기억 유지됨
```

✅ 완료 확인: CLAUDE.md 파일 생성 + /memory 저장 + /clear 후에도 ALD 조건 기억 유지

실험보고서 스킬 직접 만들기

skill-creator 설치 → 실험보고서 스킬 생성 → 호출 테스트 — 15분

```
# Step 1: skill-creator 설치
> /install-plugin skill-creator
# 설치 완료 메시지 확인
```

```
# Step 2: 스킬 생성 요청
> 반도체 공정 실험 보고서를
  자동 생성하는 스킬 만들어줘.
  실험 조건(온도/압력/시간)과
  결과 데이터를 입력받아서
  Word 파일로 출력해줘.
  '보고서 만들어줘' 말하면
  자동 실행되게 해줘.
```

```
# Step 3: 스킬 호출 테스트
> 실험보고서 만들어줘
# → 스킬 자동 트리거!
# → 실험 조건 질문
```

```
# 상세 입력 테스트:
> CVD 실험 보고서 만들어줘.
  온도: 450°C, 압력: 0.5 Torr,
  증착 시간: 30분, 두께: 120nm
```

```
# 결과: 실험보고서.docx 생성!
```

```
# Step 4: 스킬 파일 확인
$ cat .claude/skills/
  experiment-report/SKILL.md
```

✅ 완료 확인: SKILL.md 파일 생성 + '보고서 만들어줘' 입력 시 docx 파일 자동 생성

Sub-Agent 병렬 실행 체험

커스텀 에이전트 생성 → 병렬 작업 실행 → 시간 비교 — 15분

```
# Step 1: 커스텀 에이전트 만들기
> /agents
# → Create new agent 선택
# → Project-level 선택
# → Generate with Claude 선택

> 이 에이전트는 반도체 영문
  데이터시트를 한국어로 번역하는
  전문가야. 반도체 용어를 정확히
  번역하고, Read-Write 도구만
  사용해. Haiku 모델로 빠르게.

# 저장: .claude/agents/translator
```

```
# Step 2: 병렬 실행 테스트
> Sub-Agent로 다음 두 작업을
  동시에 해줘:
  1) MOSFET 데이터시트를
     translator 에이전트로 번역
  2) 번역된 내용으로 교수님께
     보낼 이메일 초안 작성

# 결과: 두 작업 동시 실행!

# Step 3: 시간 비교
# 순차 실행: ~8분
# 병렬 실행: ~3분
# → 약 2.5배 빠름!
```

✅ 완료 확인: .claude/agents/ 폴더 생성 + 번역·이메일 두 작업이 동시 실행되는 것 확인

Hook 설정 — 작업 완료 자동 알림

'Hook 만들어줘' 한 마디 → settings.json 자동 설정 — 10분

```
# Step 1: 알림 Hook 요청
> 작업이 끝나면 Windows에서
소리 알림이 오게 Hook 만들어줘.
10초간 반복 비프음으로.
```

```
# Claude가 자동으로 생성:
# ~/.claude/settings.json 에 추가
{"hooks":{"Stop":[{"matcher":"*",
  "hooks":[{"type":"command",
    "command":"powershell -c
    '1..10|%{[console]::beep(
    1000,500);Start-Sleep 0.5}'"}]}]}
```

```
# Step 2: Hook 동작 확인
> 간단한 작업 시켜서 테스트:
> '안녕'이라고 답장해줘
# → 답장 후 비프음 발생 확인!
```

```
# Step 3: PostToolUse Hook 추가
> 파일 수정할 때마다 자동으로
'echo 파일 수정됨' 출력하는
Hook도 추가해줘.
```

```
# Step 4: settings.json 확인
$ cat ~/.claude/settings.json
# → 두 Hook 모두 설정됨 확인
```

✅ 완료 확인: 작업 완료 시 소리 알림 + 파일 수정 시 메시지 출력 동작 확인

자유 실습 & 나만의 파이프라인 구성

오늘 배운 것을 조합해서 반도체 연구·현장 업무에 맞는 자동화 파이프라인 만들기 — 10분

CLAUDE.md 고도화

- › 우리 연구실 특성에 맞게 CLAUDE.md를 더 상세하게 작성해줘

스킬 추가 생성

- › 논문 요약 스킬 만들어줘.
IEEE 논문 → 한국어 3줄 요약

병렬 업무 적용

- › 실험 보고서 작성 + 데이터시트 번역을 동시에 Sub-Agent로 처리

Hook 커스터마이징

- › 실험 데이터 파일 저장 시 자동으로 백업본 만드는 Hook

오늘 만든 것 & 다음 단계

수고하셨습니다! 오늘 완성한 것들을 확인해보세요

✓ CLAUDE.md

Lazy Loading 구조
연구실 지식 베이스

✓ Skills 2종

실험보고서 자동화
업무 매뉴얼 시스템

✓ 커스텀 에이전트

번역 전문 에이전트
병렬 실행 경험

✓ Hooks 2종

완료 알림 자동화
파일 수정 감지



3회차 예고: Local LLM 서버 구축 + API 프록시 + 완전 보안 환경 — 날짜 별도 공지